

Projeto Módulo 3

Low Code / No Code /

Vibecode

João Pedro de Sousa Corrêa Teixeira - 48186368

Breno Amorim da Silva - 47126922

Bianca Moura Sena Araújo - 47717769

Cauã Spinola de Oliveira – 47399023

Gabriella Barbosa Neves - 47764961

Lucas Lima Nascimento - 48367621

01

Desafio Escolhido

A ideia do projeto surgiu de um problema real que a gente viu acontecer: corretores de plano de saúde ainda usam planilha de Excel pra fazer cotação. O corretor abre a planilha, digita as idades dos clientes, vai abrindo tabela por tabela de cada operadora, e no final tenta montar uma apresentação decente no PowerPoint. Dá muito trabalho, demora bastante, e o resultado ainda é feio.

Aí a gente decidiu fazer um sistema de cotação de planos de saúde pra uma corretora chamada Saúde Prime, aqui de Brasília. A proposta era simples: o corretor informa as idades e quantidades de pessoas, e o sistema já compara automaticamente os preços de várias operadoras ao mesmo tempo. Além disso, com um clique, ele gera uma

apresentação em PDF já formatada, com linguagem comercial, pronta pra mostrar pro cliente.

Um detalhe que a gente achou interessante de resolver: alguns clientes têm CNPJ mas teimam em querer contratar plano como pessoa física. Então o sistema precisava conseguir comparar planos de modalidades diferentes (PME, Adesão, PF) na mesma tela, o que não é trivial porque cada modalidade tem regras e faixas de preço diferentes.

02

Protótipo

O sistema ficou dividido em três arquivos que funcionam juntos:

- **Cotador principal (`cotador-planos-saude.html`):** O corretor seleciona as faixas etárias, a quantidade de vidas e a modalidade de contratação. O sistema mostra os preços de 9 operadoras em colunas: Unity, Evo, Plenum, Amil, MedSênior, Seguros Unimed, Porto Saúde, Bradesco e Best Sênior. Tem também aba de rede credenciada e aba de carências.
- **Base de dados central (`dados.js`):** Todos os preços, planos e regras de cada operadora ficam aqui. A separação foi proposital pra não ter dado espalhado pelos três arquivos. Se precisar atualizar preço, mexe só aqui.
- **Gerador de apresentação (`apresentacao-executiva.html`):** Recebe os dados do cotador e monta um PDF com capa institucional, tabela de preços detalhada por faixa etária, rede credenciada, tabela de carências de cada operadora e chamada pra ação com data de validade da proposta. Tudo gerado pelo próprio navegador, sem precisar de servidor.

O ponto mais relevante: o sistema funciona 100% offline. Dá pra jogar numa pasta no computador, sem internet, sem hospedagem, sem mensalidade de nenhuma plataforma.

Plataforma Utilizada

A plataforma escolhida foi o **Claude Code (Anthropic)**, com a abordagem de desenvolvimento chamada **Vibecode**. A escolha foi motivada por três razões práticas:

- O sistema precisava funcionar offline, ferramentas como Bubble ou Make dependem de servidores próprios. Com vibecode, o resultado é código HTML/JS puro que roda em qualquer navegador.
- As regras de negócio do mercado de saúde são específicas demais pra se encaixar em blocos visuais de no-code tradicional (diferença entre ARC 400 e ARC 500, modelo RC1/RC2/RC3 da Evo, critérios de IMC da Plenum etc.).
- A velocidade de iteração foi maior: a gente descrevia o que queria em português e o modelo gerava o código. As correções chegavam em minutos, não em horas.

Tecnologia	Uso no projeto
HTML5 / CSS3	Interface e layout das telas

JavaScript (Vanilla)

Lógica de cotação, filtros e geração de apresentação

CSS @media print

Geração de PDF via impressão do navegador

Git + GitHub

Versionamento e portfólio

Claude Code (Anthropic)

Desenvolvimento assistido por IA (Vibecode)

04

Vantagens Identificadas

- **Velocidade de desenvolvimento:** coisas que levariam dias foram feitas em horas. Quando a gente sabia exatamente o que queria, o código saía certo na primeira tentativa.
- **Zero infraestrutura:** nenhuma mensalidade, nenhum servidor, nenhum banco de dados pago. O sistema inteiro são três arquivos HTML.
- **Arquitetura emergente:** a separação entre `dados.js` e as telas surgiu durante o processo — o modelo sugeriu e fez todo sentido. Aprendemos um padrão de arquitetura real sem ter estudado formalmente.

- **Ciclo de feedback curtíssimo:** ver no navegador → identificar o problema → descrever em texto → receber a correção. Sem compilar, sem fazer deploy.
- **Código aberto e portátil:** diferente de um sistema feito em Bubble, que fica preso na plataforma, o resultado aqui é código que qualquer desenvolvedor consegue entender e manter.

05

Limitações Encontradas

- **Perda de contexto entre sessões:** as conversas foram longas e em vários dias. A cada nova sessão, era necessário contextualizar e explicar de novo a arquitetura e as decisões tomadas.
- **Ausência de testes automatizados:** nenhum teste unitário foi escrito. Os bugs só apareciam durante testes manuais com dados reais. Em produção, isso seria um risco.
- **Manutenção requer conhecimento técnico:** não tem interface de administração amigável. Atualizar preços exige editar o `dados.js` diretamente.
- **PDF via navegador é inconsistente:** cada sistema operacional e versão de Chrome se comporta de forma diferente na impressão. Foram várias iterações ajustando CSS de `@media print`.
- **Dependência da qualidade da especificação:** descrições vagas geravam código errado. Descrições precisas com contexto e exemplos reais geravam código certo de primeira.

06

Reflexão Crítica

Sobre a memória entre sessões: a solução foi criar um arquivo `MEMORY.md` com um registro das decisões técnicas tomadas, a arquitetura do sistema e o estado atual do projeto. No começo de cada sessão nova, esse contexto era colado. Foi manual, mas funcionou.

Sobre a falta de testes: adotamos uma política de testar na prática antes de commitar. Cada funcionalidade nova era exercida com dados reais, idades reais, operadoras reais, cenários com e sem ARC antes de ir pro Git.

Sobre manutenção: todo o dado de preços e operadoras foi separado no `dados.js` com estrutura documentada. A ideia é que uma atualização futura de preço possa ser feita por alguém com conhecimento básico de JavaScript.

Sobre o PDF: as inconsistências se concentravam em `@page margins` e `break-inside: avoid`. A solução foi testar especificamente no Chrome, que tem o melhor suporte a CSS de impressão.

A lição mais importante: vibecode não elimina a necessidade de entender o problema. O modelo é eficiente em traduzir intenções bem articuladas em código, mas se você não sabe o que quer, o código não vai te salvar.

Colaboração

O trabalho foi dividido em quatro frentes:

- **Requisitos:** conversar com corretores reais pra entender o fluxo atual e o que mais doía no processo manual.
- **Dados:** coletar tabelas de preços, carências e hospitais das operadoras e validar com documentação oficial.
- **Desenvolvimento:** conduzir as sessões de vibecode, revisar o código, testar funcionalidades e gerenciar o repositório Git.
- **Design/UX:** definir paleta de cores, hierarquia visual da apresentação e testar a experiência do corretor em campo.

As sessões de desenvolvimento eram feitas em dupla: Alguém do lado técnico e alguém que conhecia o domínio do negócio. Isso ajudou a pegar erros de lógica que o código gerava correto tecnicamente, mas que não faziam sentido pra quem é corretor de saúde.

09

Próximos Passos

Melhorias imediatas:

- Painel de admin simples: formulário HTML onde o corretor atualiza preços sem editar o `dados.js` na mão
- Botão de compartilhar pelo WhatsApp: integração com a API do WhatsApp Web pra enviar a apresentação diretamente
- Histórico de cotações: salvar no `localStorage` do navegador as cotações anteriores

Evoluções para o Projeto Final:

- Migrar o `dados.js` pra um banco de dados em nuvem (Supabase, plano gratuito) com API simples, atualizações de preço feitas de qualquer dispositivo
- Autenticação por corretor, com histórico individual e cálculo de comissões
- Geração de apresentação via API do Claude texto personalizado com base no perfil real de cada cliente, tornando cada proposta genuinamente única